Credit Regulator Pro Production At Scale Audit

Audit date: 2026-05-21

Repository: `C:\Users\webbd\Projects\creditregulatorpro-staging`

Audited HEAD before report artifact: `0784abc525ade21a13bce3078186a04f0584f8a0`

Runtime code commit under review before checkpoint: `f8939b4f19636413646ff6b223dddbb1411c9c1c`

Audit posture: hostile production readiness certification for compliance-critical ingestion, parsing, evidence, queueing, packet generation, autonomous remediation, and release governance.

1. Executive Summary

Overall readiness classification: **NOT READY**

This repository has meaningful production-hardening work and several strong deterministic controls, but it does not meet "Production At Scale" standards today. It more closely approximates a constrained beta or controlled pilot codebase with good regression coverage and improving operational evidence. It is not certifiable for sustained multi-user ingestion, legally sensitive evidence retention, replay-safe orchestration, or production-grade rollback governance at scale.

Confidence level: **High for repository-level findings; Medium for live production configuration durability.** I inspected the repository, workflows, tests, generated evidence, queue/orchestration code, parser and compliance paths, storage code, and operational scripts. I did not inspect `.env` or secret values by policy, so durability and production mount conclusions are based on repository-enforced guarantees rather than hidden environment assumptions.

Highest-risk findings:

1. Evidence events are mutable and can accept caller-supplied hashes. This breaks evidence-grade auditability.
2. Local document and packet artifact storage is not repository-certified as durable across container replacement, rollback, or deployment.
3. Upload ingestion now performs request-bound immediate worker execution, which reintroduces high-cost parsing and compliance work into the HTTP/SSE path.
4. Ingestion and compliance persistence contain non-transactional multi-step writes that can leave partial or deleted truth under failure.
5. Rollback workflows run validation on the workflow ref, not necessarily on the rollback target SHA.
6. Parser rule promotion can bypass regression gates through an API flag while active DB rules directly mutate canonical extraction.

Deployment safety assessment: **Not certifiable.** Deployment workflows have traceability scaffolding and smoke checks, but rollback target validation, automatic rollback, host key pinning, durable artifact guarantees, and post-checkout validation are incomplete.

Deterministic integrity assessment: **Partially strong but not production-scale safe.** The canonical parser path explicitly disables AI fallback and rejects unsupported scanned PDFs, which is a strong deterministic boundary. That boundary is weakened by mutable active parser rules, non-transactional persistence, mutable evidence events, and request-bound orchestration.

Verification Performed

Commands run during this audit:

Command	Result	Notes
`git status`	PASS	Initial tree was clean at `f8939b4...`.
`git add . && git commit -m "checkpoint before codex task"`	PASS	Required checkpoint created after evidence-generating command dirtied migration evidence files.
`git diff --check`	PASS	No whitespace errors before checkpoint.
`pnpm run test:contracts`	PASS	14 tests passed.
`pnpm run check:migrations`	PASS with warnings	Governance status partial; 18 warning-only runtime schema paths; hard deploy gate disabled.
`pnpm run check:restore-drill-evidence`	PASS limited	Template validation only; no completed restore claim.
`pnpm run check`	PASS	Build, golden path, unit, deterministic ingestion, credit regression, tradeline internal, and violation correction tests passed.
`pnpm run test:api`	PASS	313 API tests passed.
`pnpm run response:soak-check`	PASS	Exercised queue duplicate collapse, retry backlog, dead letter, stale running, overlap, replay dry-run, retention preview, and drift detection.
`pnpm run baseline:production-scale-measured -- --local`	PASS	78 requests/jobs; p95 35.32 ms in local synthetic baseline; no external provider calls.
`pnpm run packet-pdf:cache-miss-proof`	PASS	Cache-miss envelope proof refreshed.
`pnpm run operator:dashboard`	FAIL	Working tree was dirty from generated evidence; also ingest health was critical: 7 queued jobs, oldest queued age 61942 seconds.
`pnpm run ingest:worker -- --dry-run --max-jobs 1 --concurrency 1`	PASS dry-run	Confirmed a real queued local ingest job was pending; no writes performed.

Passing tests are valuable but not sufficient. Several of the most serious issues are architectural or operational-safety failures that current tests encode as expected behavior.

2. Severity-Classified Findings

P0-1. Evidence Ledger Allows Caller-Supplied Hashes, Updates, And Deletes

Severity: **P0 Critical**

Files and lines:

- `endpoints/evidence/create_POST.schema.ts:6-13`
- `endpoints/evidence/create_POST.ts:31-43`
- `endpoints/evidence/update_POST.ts:67-72`
- `endpoints/evidence/delete_POST.ts:47-58`

- `helpers/hashChain.tsx:44-75`

Exploit or failure scenario: An authenticated packet owner can create an evidence event with arbitrary `previousHash` and `currentHash`, then update or delete event rows later. The repository contains a hash-chain verifier, but the create/update/delete endpoints do not enforce append-only server-computed hash-chain semantics.

Operational impact: Evidence can be rewritten after packet generation. A packet audit trail can appear complete while no longer representing a tamper-evident ledger. This is a legal and compliance trust failure.

Likelihood: **High.** The endpoints are normal runtime surfaces, not theoretical dead code.

Recommended bounded fix: Make evidence events append-only. Compute `previousHash` and `currentHash` server-side inside a transaction using a locked last-event read. Replace update/delete behavior with superseding correction or retraction events. If raw mutation must exist for emergency repair, restrict it to admin-only audited remediation with a required reason and separate audit trail.

Affects:

- Runtime safety: Yes
- Compliance integrity: Yes
- Release governance: No
- Scale readiness: Yes
- Evidence trustworthiness: Yes

P0-2. Artifact Storage Is Not Certifiably Durable Across Deploys Or Rollbacks

Severity: **P0 Critical**

Files and lines:

- `helpers/gcsStorage.ts:15-21`
- `helpers/gcsStorage.ts:62-103`
- `helpers/documentStorage.ts:15-21`
- `helpers/documentStorage.ts:54-104`
- `docker-compose.yml:1-20`
- `docker-compose.production.yml:1-20`
- `Dockerfile:20-30`
- `.github/workflows/deploy-production.yml:307-308`
- `.github/workflows/deploy-staging.yml:330-352`

Exploit or failure scenario: Report artifacts, evidence attachments, and packet PDFs can be stored as `local:` references under `document-storage` or another runtime path. The repository compose files do not define a persistent volume, object-storage requirement, or deploy preflight proving the configured storage root survives container replacement. Deploy workflows remove/recreate containers.

Operational impact: A production deploy, rollback, host move, or container replacement can leave database rows pointing to missing report files, evidence attachments, or packet PDFs. Evidence and packet reproducibility are then broken.

Likelihood: **Medium to High.** A hidden production `.env` may point at durable host storage, but the repository does not enforce or verify that invariant.

Recommended bounded fix: Add a deploy preflight that fails closed unless document storage is either object storage or an explicit persistent host mount. Add a read/write/recreate or sentinel durability proof. Document the required storage contract and alert when `local:` artifacts reference a non-mounted path.

Affects:

- Runtime safety: Yes
- Compliance integrity: Yes
- Release governance: Yes
- Scale readiness: Yes
- Evidence trustworthiness: Yes

P1-1. Rollback SHA Validation Tests The Workflow Ref, Not The Rollback Target

Severity: **P1 High**

Files and lines:

- `.github/workflows/deploy-production.yml:46-68`
- `.github/workflows/deploy-production.yml:82-110`
- `.github/workflows/deploy-staging.yml:40-62`
- `.github/workflows/deploy-staging.yml:76-104`

Exploit or failure scenario: A `workflow_dispatch` rollback can provide `rollback_sha`, but the check job runs before target resolution and checks out the workflow ref. The workflow can pass tests for the current branch while deploying an older or different rollback target that was not revalidated in that run.

Operational impact: Rollback evidence can claim the deployment passed checks while the actual deployed SHA did not. This creates a release governance and incident-response certification gap.

Likelihood: **Medium.** It requires rollback dispatch usage, but rollback workflows are exactly where evidence accuracy matters most.

Recommended bounded fix: Resolve and validate `TARGET_SHA` before the check job. Checkout the exact target SHA for all validation and evidence generation. Fail if the target is not reachable from the approved branch or if generated evidence does not embed the target SHA.

Affects:

- Runtime safety: Yes
- Compliance integrity: No
- Release governance: Yes
- Scale readiness: Yes

- Evidence trustworthiness: Yes

P1-2. Deployment Replaces Containers Without Automatic Rollback Or Blue-Green Safety

Severity: **P1 High**

Files and lines:

- `.github/workflows/deploy-production.yml:292-308`
- `.github/workflows/deploy-production.yml:372-436`
- `.github/workflows/deploy-staging.yml:330-380`
- `.github/workflows/deploy-staging.yml:742`

Exploit or failure scenario: Deployment force-updates the remote working tree, builds, removes the running container, starts the new stack, then runs health checks. If post-deploy health fails, the workflow fails but does not automatically restore the previous known-good container, image, or SHA.

Operational impact: A bad deploy can cause an extended outage until manual recovery. Partial deploy failures can leave production in a mixed or unavailable state.

Likelihood: **Medium.** Health check failures are common under real deploy pressure, especially during migrations, network failures, or runtime dependency drift.

Recommended bounded fix: Preserve the previous image/container and SHA, deploy new version behind a readiness gate, then switch traffic only after health passes. At minimum, add an automatic rollback step that restores the prior SHA/container when health checks fail.

Affects:

- Runtime safety: Yes
- Compliance integrity: Indirect
- Release governance: Yes
- Scale readiness: Yes
- Evidence trustworthiness: Yes

P1-3. Request-Bound Immediate Ingest Processing Reintroduces High-Cost Work Into The HTTP Path

Severity: **P1 High**

Files and lines:

- `endpoints/ingest/process_POST.ts:20-22`
- `endpoints/ingest/process_POST.ts:187-224`
- `endpoints/ingest/process_POST.ts:231-280`
- `helpers/ingestProcessingQueueService.ts:1142-1199`

- `tests/api/report-ingest-lifecycle-endpoint.spec.ts:646-708`
- `tests/unit/ingest-processing-queue-boundary.spec.ts:34-62`

Exploit or failure scenario: The ingest endpoint enqueues a job, immediately claims it with a request-bound worker id, and calls the ingest process from the streaming request. A client can submit large reports or multiple concurrent uploads and force CPU, OCR, parser, canonical mapping, compliance scanning, evidence binding, and packet-readiness work into API request lifetimes.

Operational impact: The API process becomes the worker. Concurrent uploads can saturate Node, DB connections, CPU, memory, and request slots. Proxy/client disconnects can leave confusing queue state. Current tests encode this as intended behavior, so CI will not catch the production-scale regression.

Likelihood: **High.** This is the current default endpoint behavior.

Recommended bounded fix: Keep the HTTP path limited to validation, durable enqueue, and progress subscription. Use a real worker lifecycle with bounded concurrency, lease heartbeat, backpressure, and idempotent resume. If immediate processing is retained for localhost/demo, gate it behind a non-production flag that fails closed in staging/production.

Affects:

- Runtime safety: Yes
- Compliance integrity: Indirect
- Release governance: No
- Scale readiness: Yes
- Evidence trustworthiness: Indirect

P1-4. Ingest Persistence Allows Partial Writes And Replay Drift

Severity: **P1 High**

Files and lines:

- `helpers/ingestCorePipeline.tsx:335-366`
- `helpers/ingestCorePipeline.tsx:394-452`
- `helpers/ingestCorePipeline.tsx:639-678`
- `helpers/ingestCorePipeline.tsx:865`
- `helpers/comprehensiveReportStorage.tsx:51-54`
- `helpers/comprehensiveReportStorage.tsx:91-137`
- `helpers/comprehensiveReportStorage.tsx:312-518`

Exploit or failure scenario: The ingest pipeline writes artifacts, extraction snapshots, replay payloads, parsed tradelines, payment history, compliance scans, violation review runs, and final status across many independent operations. Several storage branches intentionally catch/log/continue on failure. A crash or DB error can leave mixed old/new state while the report is later marked complete or retried.

Operational impact: Replay may not reproduce the exact persisted truth. Downstream evidence references can point at incomplete artifact sets. Operators may see a successful report with missing consumer information, payment history,

compliance findings, or replay data.

Likelihood: **Medium to High.** Multi-step persistence under load or malformed input is a normal production failure class.

Recommended bounded fix: Define transaction boundaries for each durable state transition. Persist parser output, canonical mapping, evidence indexes, and compliance results as an atomic stage record before promotion to active report state. Treat optional non-critical persistence separately with explicit degraded state, not silent continuation.

Affects:

- Runtime safety: Yes
- Compliance integrity: Yes
- Release governance: No
- Scale readiness: Yes
- Evidence trustworthiness: Yes

P1-5. Compliance Findings Can Be Deleted Before Replacement Inserts Succeed

Severity: **P1 High**

Files and lines:

- `helpers/complianceScanner.tsx:625-810`
- `helpers/complianceScanner.tsx:661-693`
- `helpers/complianceScanner.tsx:699-785`
- `helpers/complianceScanner.tsx:781-784`

Exploit or failure scenario: `persistViolations` preserves non-active manual/admin findings, deletes active auto-generated findings, then inserts replacement findings one-by-one while catching insert failures. A failure after the delete can remove previously active findings and persist only a partial replacement set.

Operational impact: Confirmed or reviewable compliance signals can disappear or become inconsistent. Packet readiness can change based on partial writes rather than deterministic rule output.

Likelihood: **Medium.** It depends on DB/storage failure or concurrent scans, but this is exactly the failure mode a production compliance scanner must survive.

Recommended bounded fix: Wrap delete and insert in a transaction. Insert replacement findings into a staging set and swap active set atomically. Preserve the previous active set if the new scan cannot be fully persisted.

Affects:

- Runtime safety: Yes
- Compliance integrity: Yes
- Release governance: No
- Scale readiness: Yes
- Evidence trustworthiness: Yes

P1-6. Parser Rule Promotion Can Bypass Regression Gates While Mutating Canonical Truth

Severity: **P1 High**

Files and lines:

- `helpers/parserExtractionRules.tsx:200-290`
- `helpers/canonicalCreditReportExtractor.tsx:405-424`
- `endpoints/parser-test-case/promote-rule_POST.schema.ts:6`
- `endpoints/parser-test-case/promote-rule_POST.ts:345`
- `endpoints/parser-test-case/promote-rule_POST.ts:439-458`
- `endpoints/parser-test-case/promote-rule_POST.ts:510-529`

Exploit or failure scenario: Active parser extraction rules are loaded from the database and mutate canonical extraction results. The promotion endpoint permits `runRegressionGate` in input and can skip before/after full regression when false. The UI appears to send true, but the API trust boundary permits false.

Operational impact: An admin or compromised admin session can change deterministic parser truth without mandatory regression evidence. This violates the "No Silent Truth Change" requirement.

Likelihood: **Medium.** Requires admin/API access, but admin correction paths are production-critical.

Recommended bounded fix: Reject `runRegressionGate: false` in non-test environments unless a separate break-glass role, reason, and audit event are provided. Store gate evidence with the promoted rule and refuse activation without a passing gate.

Affects:

- Runtime safety: Yes
- Compliance integrity: Yes
- Release governance: Yes
- Scale readiness: Yes
- Evidence trustworthiness: Yes

P1-7. Packet PDF Cache-Miss Timeout Does Not Cancel Underlying Render Work

Severity: **P1 High**

Files and lines:

- `helpers/packetPdfCache.ts:162-227`
- `helpers/packetPdfCacheMissEnvelope.ts:3-5`
- `helpers/packetPdfCacheMissEnvelope.ts:152-169`
- `helpers/packetPdfCacheMissEnvelope.ts:195-211`

Exploit or failure scenario: Cache-miss rendering is wrapped in a timeout and concurrency envelope, but the timeout rejects the caller without aborting the original render promise. The slot is released in `finally`, so timed-out render work can continue while new work enters the envelope.

Operational impact: Repeated PDF cache misses can accumulate CPU/memory work behind timed-out requests. Under scale, packet rendering can exceed the advertised concurrency protection and degrade the API process.

Likelihood: **Medium.** Large packet PDFs and concurrent downloads are realistic production behavior.

Recommended bounded fix: Use an abortable renderer or out-of-process worker. If cancellation is unavailable, hold the concurrency slot until the underlying render promise settles and expose a degraded async-render state instead of retrying inline.

Affects:

- Runtime safety: Yes
- Compliance integrity: Indirect
- Release governance: No
- Scale readiness: Yes
- Evidence trustworthiness: Indirect

P1-8. Current Queue State Shows Critical Staleness And No Drained-Queue Proof

Severity: **P1 High**

Files and lines:

- `scripts/operator-dashboard.ts`
- `scripts/ingest-processing-worker.ts:485-607`
- `scripts/ingest-processing-worker.ts:609-650`
- `helpers/ingestProcessingQueueService.ts:1075-1199`

Exploit or failure scenario: `pnpm run operator:dashboard` reported ingest health failure with 7 queued jobs and an oldest queued age of 61942 seconds. A dry-run worker preview confirmed a queued `report_ingest_process` job was pending. The worker supports bounded dry-run/application behavior, but this audit did not find an active sustained worker proof.

Operational impact: Queue state can appear durable but remain operationally stuck. Operators may need manual intervention to drain jobs, and users may observe stale "queued" or "processing" states.

Likelihood: **High in the audited local environment.** The production environment may differ, but the repository evidence does not prove sustained worker liveness.

Recommended bounded fix: Add an operator-grade queue liveness proof: active worker heartbeat, oldest queued age SLO, failed/dead-letter dashboard, and deploy gate or release evidence requiring queue freshness before promotion.

Affects:

- Runtime safety: Yes
- Compliance integrity: Indirect
- Release governance: Yes
- Scale readiness: Yes
- Evidence trustworthiness: Indirect

P2-1. SSH Host Key Trust Uses Runtime `ssh-keyscan`

Severity: **P2 Medium**

Files and lines:

- `.github/workflows/deploy-production.yml:200`
- `.github/workflows/deploy-staging.yml:194-220`

Exploit or failure scenario: Deployment workflows populate `known_hosts` with `ssh-keyscan` at runtime. This is trust-on-first-use in CI. A network-level attacker during deploy setup can present a malicious host key.

Operational impact: A compromised deploy SSH trust boundary can redirect deployment commands or expose deployment behavior.

Likelihood: **Low to Medium.** It requires network/DNS/MITM conditions, but deployment infrastructure warrants stronger controls.

Recommended bounded fix: Store the expected host key or fingerprint in GitHub environment variables/secrets and verify `ssh-keyscan` output against it before adding to `known_hosts`.

Affects:

- Runtime safety: Yes
- Compliance integrity: No
- Release governance: Yes
- Scale readiness: Yes
- Evidence trustworthiness: Yes

P2-2. Remote Deployment Mutates Working Tree And Staging Lacks Post-Checkout SHA Verification

Severity: **P2 Medium**

Files and lines:

- `.github/workflows/deploy-production.yml:302`
- `.github/workflows/deploy-staging.yml:307-309`
- `.github/workflows/deploy-production.yml:295-301`

Exploit or failure scenario: Production copies `docker-compose.production.yml` to `docker-compose.yml` on the remote host, mutating the deploy checkout. Staging checks out `TARGET_SHA` but does not perform the same explicit `git rev-parse HEAD` equality check production performs.

Operational impact: Remote state can drift from repository state, and staging deployment evidence can be weaker than production deployment evidence.

Likelihood: **Medium.** Remote drift is common over repeated deployments.

Recommended bounded fix: Use `docker compose -f docker-compose.production.yml` instead of copying files. Add post-checkout SHA verification to staging identical to production.

Affects:

- Runtime safety: Indirect
- Compliance integrity: No
- Release governance: Yes
- Scale readiness: Yes
- Evidence trustworthiness: Yes

P2-3. Bureau Communication Attachment Hashes Base64 Text, Not Decoded Bytes

Severity: **P2 Medium**

Files and lines:

- `endpoints/evidence/bureau-communication_POST.ts:236`
- `endpoints/evidence/bureau-communication_POST.ts:239-245`
- `endpoints/evidence/bureau-communication_POST.ts:263-300`
- `helpers/reportBinaryUtils.tsx:1-18`

Exploit or failure scenario: Attachment evidence hashes `input.fileDataBase64` as a string. Different base64 encodings or data URL formatting for the same binary can produce different evidence hashes. The repository already has a decoded-byte SHA-256 helper used elsewhere.

Operational impact: Operators cannot reliably compare attachment hashes to the underlying PDF/image bytes. Evidence may appear inconsistent across upload paths or external forensic tools.

Likelihood: **Medium.** Base64 formatting differences are common.

Recommended bounded fix: Use the decoded-byte hash helper for bureau communication attachments and store both MIME metadata and byte digest. Add a regression test proving equivalent base64 encodings produce the same hash.

Affects:

- Runtime safety: No
- Compliance integrity: Yes

- Release governance: No
- Scale readiness: No
- Evidence trustworthiness: Yes

P2-4. Migration Governance Remains Partial And Runtime Schema Ensures Mask Drift

Severity: **P2 Medium**

Files and lines:

- `docs/production-scale/evidence/latest-migration-governance.md`
- `scripts/check-migration-governance.ts`
- `helpers/parserRulePromotionSchema.tsx:6-73`

Exploit or failure scenario: `pnpm run check:migrations` passed but reported governance status partial, 18 warning-only runtime schema paths, and hard deploy gate disabled. Runtime schema ensure paths can create or alter tables at runtime, which masks migration drift until production behavior diverges.

Operational impact: Schema changes can be applied implicitly rather than through reviewed migrations and rollback-aware release governance. This weakens recovery and drift diagnosis.

Likelihood: **High.** The evidence command explicitly reports partial governance.

Recommended bounded fix: Convert runtime ensure paths into reviewed additive migrations one subsystem at a time. Keep warning evidence visible during cutover, then enable a hard migration drift gate for production deploys.

Affects:

- Runtime safety: Yes
- Compliance integrity: Yes
- Release governance: Yes
- Scale readiness: Yes
- Evidence trustworthiness: Yes

P2-5. Evidence Hash Chain Semantics Are Inconsistent Across Packet And General Evidence Paths

Severity: **P2 Medium**

Files and lines:

- `helpers/disputePacketService.ts:1457-1468`
- `endpoints/evidence/create_POST.ts:31-43`
- `helpers/hashChain.tsx:44-75`

Exploit or failure scenario: Packet generation creates an evidence event with `previousHash: null` and `currentHash: hashEvent(eventData)`, while general evidence creation accepts caller-provided hashes. The verifier expects chain semantics, but producers do not consistently maintain a single append-only chain.

Operational impact: Hash-chain verification can give a false sense of tamper evidence. Separate event creation paths produce incompatible chain behavior.

Likelihood: **High.** Both paths exist in active code.

Recommended bounded fix: Centralize evidence-event creation through a single append-only service that computes chain links, validates packet/report ownership, and emits corrections as new events.

Affects:

- Runtime safety: No
- Compliance integrity: Yes
- Release governance: No
- Scale readiness: Yes
- Evidence trustworthiness: Yes

P2-6. Operational Evidence Is Stale Or Explicitly Non-Certifying In Multiple Places

Severity: **P2 Medium**

Files and lines:

- `docs/production-scale/evidence/latest-production-promotion-pack.md`
- `docs/production-at-scale-maximum-audit.md`
- `docs/production-scale/FINAL_VERIFICATION.md`
- `scripts/operator-dashboard.ts`

Exploit or failure scenario: Existing promotion and audit documents reference older commits and classify readiness below production-at-scale. The operator dashboard includes skipped, open, and human-required checks, and during this audit it failed ingest health.

Operational impact: Stakeholders can mistake stale generated evidence for current release certification. Release evidence can drift from the code actually being deployed.

Likelihood: **High.** The stale documents are present in-repo and easy to cite.

Recommended bounded fix: Make production promotion evidence include current HEAD, target environment, exact command set, evidence freshness timestamp, queue liveness, and an explicit "certifying/not certifying" flag. Fail promotion when evidence is stale relative to target SHA.

Affects:

- Runtime safety: Indirect
- Compliance integrity: Indirect

- Release governance: Yes
- Scale readiness: Yes
- Evidence trustworthiness: Yes

P2-7. Pull Request Regression Guardrail Is Too Narrow For A Compliance-Critical Platform

Severity: **P2 Medium**

Files and lines:

- `.github/workflows/pr-regression-guardrails.yml:1-31`

Exploit or failure scenario: The PR guardrail workflow runs the golden path, but not the broader API, contracts, migration governance, queue soak, packet PDF cache-miss proof, or deterministic ingestion suite.

Operational impact: Risky changes can merge after passing the happy path while breaking API boundaries, queue behavior, evidence governance, or packet rendering envelope.

Likelihood: **Medium.** The repository has many specialized tests that are not all represented in this guardrail.

Recommended bounded fix: Keep the golden path fast but add a second required workflow for contracts, API tests, deterministic ingestion, migration governance, and packet PDF cache proof. Reserve full soak for scheduled or pre-promotion runs if runtime is too high for every PR.

Affects:

- Runtime safety: Yes
- Compliance integrity: Yes
- Release governance: Yes
- Scale readiness: Yes
- Evidence trustworthiness: Yes

P2-8. Admin Correction Finalization Performs Multi-Step Truth Changes Without A Single Transaction

Severity: **P2 Medium**

Files and lines:

- `helpers/violationCorrectionManager.tsx:358-405`

Exploit or failure scenario: Review finalization updates correction status, upserts parser training material, and builds additional metadata across multiple operations. A failure after status update can leave correction state and downstream training/audit artifacts out of sync.

Operational impact: Admin corrections can become canonical in one table while related training or review metadata is incomplete. This weakens the human review trail.

Likelihood: **Medium.** Admin correction flows are less frequent than ingestion but high impact.

Recommended bounded fix: Wrap finalization state changes and associated training/audit writes in a transaction. If some work is intentionally asynchronous, store an explicit pending/degraded stage and require operator visibility.

Affects:

- Runtime safety: Yes
- Compliance integrity: Yes
- Release governance: No
- Scale readiness: Yes
- Evidence trustworthiness: Yes

P2-9. Upload UI Communicates Queue State, But Backend Liveness Is Not Guaranteed

Severity: **P2 Medium**

Files and lines:

- `pages/upload.tsx:44-155`
- `pages/upload.tsx:224-306`
- `pages/upload.tsx:491-639`
- `endpoints/ingest/process_POST.ts:231-280`

Exploit or failure scenario: The upload page has detailed queued, processing, stale, failed, and check-status messaging. However, backend processing can be request-bound or stuck in a stale queue. The UI can ask users to wait or check status when no active worker guarantee exists.

Operational impact: Users may see ambiguous progress states and retry uploads, causing duplicate pressure and support load.

Likelihood: **Medium.** Confirmed stale queue state exists in the audited local environment.

Recommended bounded fix: Surface backend worker liveness and queue age in user-safe language. When no worker heartbeat exists, fail closed with a support/remediation state instead of continuing optimistic progress.

Affects:

- Runtime safety: Indirect
- Compliance integrity: No
- Release governance: No
- Scale readiness: Yes
- Evidence trustworthiness: No

P3-1. Frontend Bundle Size Is Large And Not A Hard Gate

Severity: **P3 Low**

Files and lines:

- `package.json`
- `vite.config.ts`

Exploit or failure scenario: The production build generated a main JavaScript bundle around 3.29 MB raw and 902.52 kB gzip, plus CSS around 690 kB raw and 105.68 kB gzip. This is not currently a release-blocking issue.

Operational impact: Slower first loads and greater risk of degraded UX under mobile or constrained networks.

Likelihood: **Medium.** The size is measurable in the current build.

Recommended bounded fix: Add bundle reporting and route-level code splitting for admin/parser-lab/heavy PDF surfaces. Keep this as a performance budget, not a blocker until runtime evidence shows user impact.

Affects:

- Runtime safety: No
- Compliance integrity: No
- Release governance: No
- Scale readiness: Yes
- Evidence trustworthiness: No

P3-2. Packet List Query Uses Client-Side Filtering For Ownership/Organization Context

Severity: **P3 Low**

Files and lines:

- `helpers/packetQueries.tsx:24-31`

Exploit or failure scenario: Client-side logic filters packet lists returned by the endpoint for user/org scope. This is not direct evidence of authorization bypass, but it is a scale and trust-boundary smell if server-side scope is not guaranteed elsewhere.

Operational impact: Larger packet volumes may overfetch data and increase UI latency. Future maintainers may mistake client filtering for an authorization boundary.

Likelihood: **Low to Medium.**

Recommended bounded fix: Ensure server-side packet list APIs enforce user/org scope and pagination, then keep client filtering only as a defensive display guard.

Affects:

- Runtime safety: Indirect

- Compliance integrity: No
- Release governance: No
- Scale readiness: Yes
- Evidence trustworthiness: No

3. Production Scale Gaps

Missing or weak systems:

- No repository-enforced durable artifact storage contract for `local:` report, evidence, and packet artifacts.
- No append-only server-computed evidence ledger.
- No deploy target validation before rollback checks.
- No automatic rollback or blue-green deployment safety.
- No sustained worker liveness proof tied to deploy gates or operator evidence.
- No hard migration drift gate yet; governance remains partial.
- No mandatory parser-rule regression gate at the API trust boundary.
- No end-to-end proof that queue workers remain active under concurrent multi-user uploads.
- No hard evidence freshness gate that prevents stale promotion packs from being used as certification.
- No cancellation-safe packet PDF render isolation.

Non-scalable assumptions:

- Request-bound ingestion assumes expensive parser/orchestration work can complete inside an API/SSE request.
- Packet PDF rendering still happens inline on cache miss.
- Some client surfaces rely on post-fetch filtering and large bundled admin/parser functionality.
- Local measured performance is a small synthetic baseline, not a high-concurrency saturation or soak proof.

Operational blind spots:

- Current operator dashboard can show stale queued ingest jobs.
- Restore drill evidence validates templates, not an actual completed restore.
- Release evidence can reference old commits.
- Migration runtime ensures are tolerated as warnings, which can hide drift until incident time.

Governance weaknesses:

- Rollback dispatch can deploy a SHA that was not the SHA tested by the check job.
- PR guardrails do not cover all high-risk suites.
- SSH deploy host trust is not pinned.
- Production remote working tree is mutated during deploy.

4. False Readiness Claims

Areas that appear production-ready but are not certifying:

1. **Passing** `pnpm run check`: Strong regression signal, but it does not cover evidence mutability, durable artifact storage, rollback target validation, or sustained queue liveness.
2. **Golden path green**: Valuable happy-path proof, but request-bound processing and partial persistence hazards remain.
3. **Production promotion pack**: The current in-repo pack references older commits and includes many reference-required or non-certifying rows. It is not current certification for this HEAD.
4. **Packet PDF cache-miss proof**: Shows timeout/envelope behavior but does not prove underlying render cancellation.
5. **Migration governance pass**: The check passes while explicitly reporting partial governance, warning-only runtime schema paths, and hard deploy gate disabled.
6. **Operator dashboard**: The dashboard is useful, but it failed during this audit and includes open/human-required checks.
7. **Deterministic parser claims**: Core AI isolation is strong, but active DB parser rules can mutate canonical extraction and can be promoted without mandatory regression gates.
8. **Hash-chain helper presence**: The verifier exists, but event creation/update/delete paths do not enforce append-only hash-chain semantics.

5. Deterministic Safety Review

Deterministic ingestion status: **Partially deterministic, not fully production-safe.**

Strong controls verified:

- Canonical extraction calls parser logic with AI augmentation disabled in `helpers/canonicalCreditReportExtractor.tsx:342-348`.
- AI fallback is marked skipped/non-eligible in canonical provenance at `helpers/canonicalCreditReportExtractor.tsx:382-391` and `helpers/canonicalCreditReportExtractor.tsx:484-503`.
- OCR fallback is deterministic and bounded; unsupported scanned PDFs are rejected through eligibility checks.
- Legacy DocStrange/LLM reparse paths are disabled in `helpers/unifiedExtractor.tsx` and `helpers/tradelineReparseSync.tsx`.
- `pnpm run test:deterministic-ingestion-report` passed with replay stable and required evidence coverage at 100 percent.

Unsafe or incomplete controls:

- Active parser extraction rules are DB-driven and can mutate canonical results.
- Parser rule promotion permits regression-gate bypass through API input.
- Ingest persistence is not stage-atomic across canonical mapping, evidence indexing, compliance scanning, replay data, and final status.
- Compliance finding persistence can delete prior active findings before new insert completion.
- Evidence rows are mutable and hash-chain semantics are not enforced by a single append-only service.

Assessment:

The parsing computation itself is substantially more deterministic than the surrounding platform. The production-scale risk is not primarily hidden LLM dependence. The risk is that deterministic output can be changed, partially persisted, or evidenced unsafely by surrounding mutable state and orchestration paths.

6. Deployment Certification

Production deploys are certifiable: **No.**

Reasons:

- Rollback target SHA is not the SHA validated by the check job.
- No automatic rollback restores the prior known-good deployment after health failure.
- Durable artifact storage is not repository-certified.
- SSH host keys are collected with runtime `ssh-keyscan` instead of pinned verification.
- Production deploy mutates the remote checkout by copying compose files.
- Evidence packs can be stale relative to the deployed SHA.

Rollback handling is safe: **No.**

Rollback dispatch exists and includes guardrails, but target validation and automatic recovery are incomplete. A failed post-deploy health check does not restore the previous version by default.

Evidence traceability is trustworthy: **No for certification; partially useful for diagnostics.**

The repository has useful generated evidence, but freshness is inconsistent and evidence ledger mutability is a P0 blocker.

Release governance is production-grade: **No.**

Governance is improving and test coverage is broad, but deploy, rollback, migration, evidence freshness, and PR gates are not yet production-grade for a compliance-critical system.

7. Top 10 Priority Fixes

1. Make evidence events append-only with server-computed hash-chain links; remove caller-supplied hashes and raw update/delete paths.
2. Enforce durable artifact storage with a deploy preflight and persistent mount or object-storage proof.
3. Move ingestion execution out of the request path for staging/production; require worker liveness, heartbeat, bounded concurrency, and queue freshness evidence.
4. Wrap compliance finding replacement in a transaction or staged atomic swap.
5. Add atomic ingest stage persistence for canonical extraction, evidence indexes, replay payload, compliance results, and final status.
6. Resolve rollback target SHA before validation and test the exact SHA that will deploy.
7. Add automatic rollback or blue-green deployment behavior on failed health checks.
8. Deny parser-rule promotion without mandatory regression evidence outside explicit audited break-glass.

9. Convert runtime schema ensures into migrations and enable a hard migration drift gate for production deploys.
10. Make packet PDF cache-miss rendering cancellation-safe or isolate it in a bounded worker.

8. Optional Safe Improvements

- Pin SSH host fingerprints in deploy workflows.
- Add staging post-checkout SHA equality verification.
- Stop copying production compose files over `docker-compose.yml`; pass the compose file explicitly.
- Add a required PR workflow for contracts, API tests, deterministic ingestion, migration governance, and packet PDF cache proof.
- Add queue dashboard rows for active worker heartbeat, oldest queued age, and dead-letter counts.
- Add decoded-byte hashing tests for all evidence attachment upload paths.
- Add a bundle-size report and split admin/parser/PDF-heavy routes.
- Add a "not certification evidence" banner to stale or partial generated evidence documents.

9. Candid Final Assessment

Credit Regulator Pro has many signs of serious engineering discipline: deterministic parser isolation, broad regression coverage, queue soak scripts, packet readiness blockers, and release evidence scaffolding. Those are not enough for "Production At Scale" certification.

The current repository does not genuinely meet production-at-scale standards. It approximates them operationally in selected golden paths, but core auditability, artifact durability, request/worker isolation, persistence atomicity, rollback certification, and evidence freshness remain below the bar for a compliance-critical platform.

The most important distinction is this: the deterministic parser core is not the weakest point. The weakest points are the mutable evidence ledger, non-certified artifact storage, request-bound orchestration, non-atomic state transitions, and release governance gaps around rollback and stale evidence. Those must be fixed before the system can credibly move beyond limited beta or controlled pilot operation.